



UNIVERSIDAD DE GRANADA
ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA
INFORMÁTICA Y TELECOMUNICACIONES

MÁSTER
INGENIERÍA INFORMÁTICA
SEGUNDO CUATRIMESTRE

SISTEMAS INTELIGENTES PARA LA GESTIÓN
DE LA EMPRESA

Práctica 1

Pablo Luque Moreno

DNI: 31012686S

pabloluquem@correo.ugr.es

Curso académico 2019-2020
Granada, 13 de abril de 2020

Índice

1. Introducción	1
2. Exploración	1
3. Pre-procesamiento	3
3.1. Selección de características	5
4. Clasificación	5
5. Discusión de resultados	5
6. Conclusiones	5

1. Introducción

En esta memoria se describe de manera no exhaustiva los pasos y técnicas empleados por el alumno que se pedían para la realización de esta práctica. El dataset pertenece a una competición de Kaggle ya cerrada sobre la detección del fraude. El *dataset* empleado se encuentra en <https://www.kaggle.com/c/ieee-fraud-detection/data>. Acerca de este dataset, previo a cualquier exploración, se sabe de antemano que cuenta con 434 variables en total, por lo que el primer desafío será aplicar las técnicas adecuadas para reducir su dimensionalidad y operar con mayor facilidad sobre los datos.

2. Exploración

Antes de realizar cualquier operación sobre el dataset se hará una exploración del mismo para conocer con más detalle el problema al que se enfrenta. En primer lugar para ver el balanceo de clases usamos un histograma.

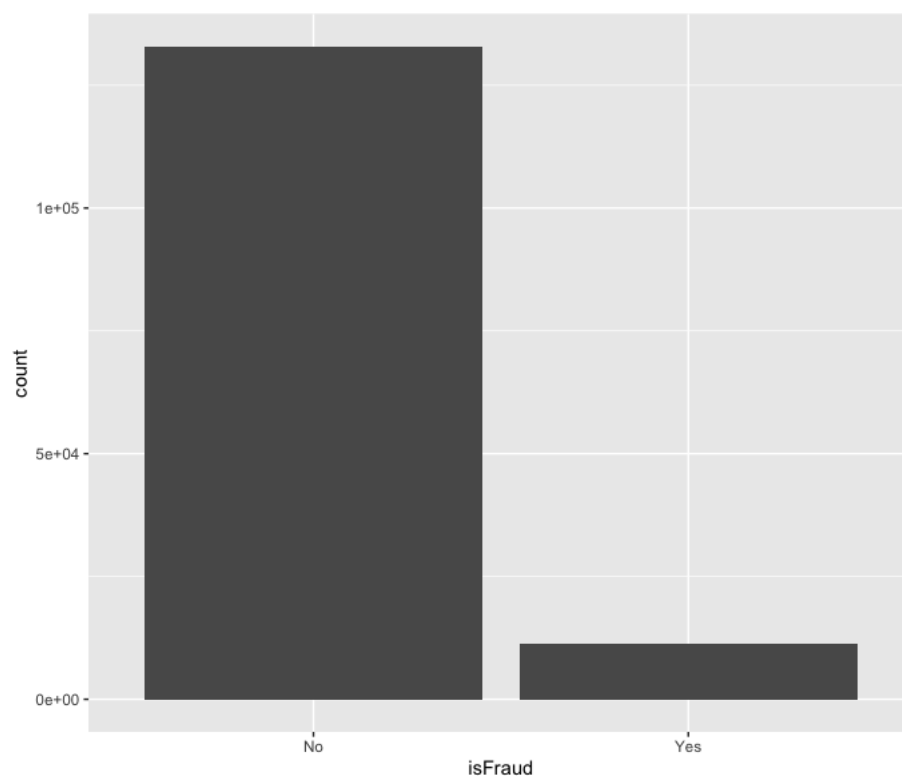


Figura 1: Histograma de las clases

Como puede verse en la figura 1 las clases de por si están muy desbalanceadas, habiendo lógicamente muchas menos transacciones fraudulentas que no fraudulentas. A continuación se realiza un box plot para ver la distribución de la cantidad de dinero en cada una de las clases.

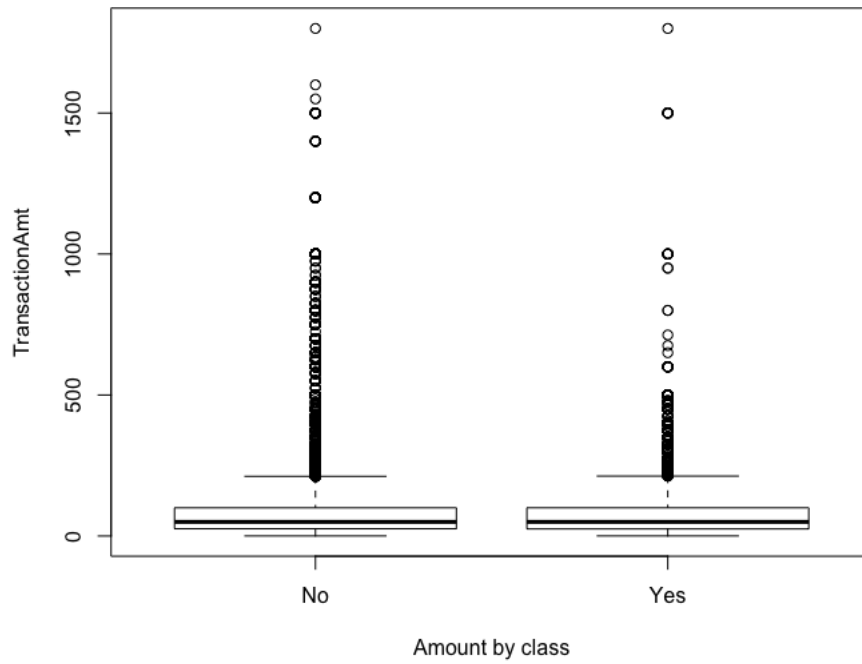


Figura 2: Boxplot

Como puede verse en la figura 2 la distribución de dinero en los dos tipos de transacciones (fraudulentas y no fraudulentas) es bastante de irregular conteniendo también *outliers*.

3. Pre-procesamiento

En primer lugar se ha categorizado las variables mencionadas en la página de kaggle para no confundirlas con variables numéricas. Esto ayudará a eliminarlas después si no son relevantes. Se muestra a continuación el código:

```
# read train data
data = read.csv(file = "data/datasets_join/train_innerjoin.csv"
)

# factorize dataset features
data_factorized <- data %>% mutate(isFraud = as.factor(ifelse(
isFraud == 1, "Yes", "No")))
data_factorized <- data_factorized %>% mutate_at(c('addr1', '
addr2'), factor)
data_factorized <- data_factorized %>% mutate_at(vars_select(
names(data_factorized), starts_with('M',)), factor)
data_factorized <- data_factorized %>% mutate_at(vars_select(
names(data_factorized), starts_with('card',)), factor)
data_factorized <- data_factorized %>% mutate_at(vars_select(
names(data_factorized),

matches("id_(1[2-9]|2[0-9]|3[0-8]))"), factor)
```

Figura 3: Código factorización

A continuación es importante borrar todas aquellas variables que tengan gran cantidad de valores nulos pues van a ser las primeras que aporten poca información. En este caso se ha decidido borrar todas las variables cuyos valores nulos superen más del 40 % del total. Se muestra a continuación el código:

```
# delete features with NAs
my_data_status=df_status(data_factorized)
vars_to_remove=subset(my_data_status, my_data_status$p_na > 40)
data_cleaned = data_factorized[,!(names(data_factorized) %in%
vars_to_remove[, "variable"])]
```

Figura 4: Código borrar valores nulos

Se ha borrado también todas las variables que presenten una alta dispersión en sus valores. La medida usada es *unique* que proporciona *df_status* la cual permite saber cuantos valores únicos tiene cada variable. Entonces se ha decidido borrar todas aquellas que tengan más de un 70 % de valores únicos frente al total. Se muestra a continuación el código:

```
# delete features with high dispersity
my_data_status=df_status(data_cleaned)
vars_to_remove=subset(my_data_status, my_data_status$unique >
(0.7 * nrow(data_cleaned)))
data_cleaned2 = data_cleaned[,!(names(data_cleaned) %in%
vars_to_remove[, "variable"])]
```

Figura 5: Código borrar valores con mucha dispersión

Por otra parte se han borrado variables con poca dispersión pues a parte de aportar poca información entorpecen el cálculo de la matriz de correlación. El método empleado es el mismo que para el caso anterior pero esta vez borrando solo aquellas variables con menos valores únicos que el 1% del total. Se muestra a continuación el código:

```
# delete features with low dispersity -> important to
correlation matrix
my_data_status = df_status(data_cleaned2)
vars_to_remove=subset(my_data_status, my_data_status$unique <
(0.01 * nrow(data_cleaned2)) & (my_data_status$type == "numeric"
| my_data_status$type == "integer"))
data_cleaned3 = data_cleaned2[,!(names(data_cleaned2) %in%
vars_to_remove[, "variable"])]
```

Figura 6: Código borrar valores con poca dispersión

Se ha borrado las variables con gran cantidad de ceros pues tampoco aportan mucha información, de tal forma que se han eliminado las variables con un 90% de ceros o más entre sus valores. Se muestra a continuación el código:

```
my_data_status=df_status(data_cleaned3)
vars_to_remove=subset(my_data_status, my_data_status$p_zeros >
90)
data_cleaned4 = data_cleaned3[,!(names(data_cleaned3) %in%
vars_to_remove[, "variable"])]
```

Figura 7: Código borrar valores con muchos ceros

Por último se ha calculado la matriz de correlación para así borrar las variables altamente correladas entre sí.

3.1. Selección de características

Para seleccionar las características más importantes de entre las restantes, se ha usado el algoritmo de random forest con el paquete *randomForest* para calcular la importancia de cada variable y así seleccionar las más importantes. Se ha modificado algunos parámetros como el número de árboles generados para no demorar demasiado el cálculo. Finalmente se seleccionan las 10 variables más importantes para aplicar los cálculos pertinentes en función del accuracy.

4. Clasificación

Pendiente de acabar

5. Discusión de resultados

Pendiente de acabar

6. Conclusiones

Pendiente de acabar